

🛠 ИНСТРУКЦИЯ ПО РАСШИРЕННОМУ ИБ-ОТЛАДОЧНОМУ СКАНИРОВАНИЮ ФРОНТЕНДА (DEVTOOLS)

Настоящее техническое руководство содержит пошаговый регламент отладки DOM-периметра, перехвата скрытых ошибок JS-рантайма в консоли браузера, трассировки сетевых Ajax-запросов (fetch) и анализа ИБ-заheaders.

КОНТУР СКВОЗНОЙ ОТЛАДКИ И ПЕРЕХВАТА AJAX-ТРАНЗАКЦИЙ (DEVTOOLS MAP)

Мониторинг аномалий фронтенда (Vanilla JS) опирается на трехуровневую фильтрацию событий: вкладка Console (ошибки рантайма), вкладка Network (XHR/fetch запросы) и инспекция заголовков.

--- Схема прохождения сетевых пакетов и фильтрации трассировок UI ---

СБОЙ ИНТЕРФЕЙСА ИЛИ АНОМАЛИЯ В auth.js / rights.js / servers.js

↓ Перехватчик браузера: Нажатие клавиши F12 —> Открытие Chrome DevTools

ЭТАП А: АНАЛИЗ ОШИБОК И СТАТУСОВ ОТВЕТОВ В СЕТИ (NETWORK)

Фильтр: Fetch/XHR —> Анализ Payload и Response JSON от FastAPI-бэкенда

↓ Изоляция сбоев внутри блоков try { ... } catch (err)

Контур А: Ошибки Синтаксиса / Импорта

- 🔥 Выброс Uncaught SyntaxError / ReferenceError.
- ✂ Сбой связывания модулей при отсутствии ключевых слов export.
- 🔪 Локализация: Анализ точной строки анонимного вызова (index):38.

Контур Б: Аномалии CORS и Секретов

- ⚠ Блокировка ответов из-за отсутствия CORS заголовков.
- 🔒 Падение сессий при вымывании JWT из localStorage.
- 🕒 Контроль ответов 401 Unauthorized и 403 Forbidden.

АНАЛИЗ ЗАВЕРШЕН —> СБРОС КЭША БРАУЗЕРА КОМБИНАЦИЕЙ CTRL + F5

🔧 Пошаговый алгоритм проведения глубокой отладки (Debugging) фронтенда:

1. **Инициализация консоли отладки DevTools:** Откройте веб-интерфейс панели в браузере Chrome или Яндекс.Браузер. Нажмите клавишу **F12** (или сочетание **Ctrl + Shift + I**) и переключитесь на вкладку **Console**. Здесь отображаются все критические сбои парсинга JavaScript.

2. **Запуск живого перехвата сетевых пакетов (Network Trace):** Перейдите на вкладку **Network** и включите фильтр **Fetch/XHR**. Сделайте тестовое действие (например, попытку авторизации или накат прав на таблицу). Нажмите на появившуюся строку запроса и изучите вкладки:

- **Payload:** Что именно ваш JS-код сформировал и отправил на бэкэнд.
- **Response:** Точный сырой JSON-ответ сервера, включая скрытые детали ошибок Pydantic или PostgreSQL.

3. **Снайперский сброс агрессивного кэширования статики:** Поскольку браузеры намертво кэшируют локальные скрипты `auth.js` и `rights.js`, любые ваши правки в коде на сервере могут игнорироваться клиентом. При открытой консоли DevTools зажмите правую кнопку мыши на значке перезагрузки страницы в браузере и выберите пункт **«Очистка кэша и жесткая перезагрузка»** (или нажмите **Ctrl + F5**).

ИСПРАВЛЕННАЯ СЛУЖЕБНАЯ ИНСТРУКЦИЯ ПО ОПЕРАТИВНОМУ ДЕБАЖУ UI

Контур обеспечивает экспресс-диагностику сбоев DOM-модели, нарушений импорта ESM-модулей, проблем локального кэширования переменных и аномалий кодировки на стороне клиента.

--- Матрица диагностических кодов и ИБ-затворов фронтенда ---

СИСТЕМНЫЙ МАРКЕР ОШИБКИ И СТАТУС В БРАУЗЕРЕ (DEVTOOLS)	
 Uncaught ReferenceError: submitLoginForm is not defined at HTMLFormElement.onSubmit	Причина: Скрипт <code>auth.js</code> запер функцию внутри изолированного модуля, или она не экспортирована во внешний контекст.  Фикс: Внедрить в конец <code>auth.js</code> глобальный ИБ-мост: <code>window.submitLoginForm = submitLoginForm;</code>
 Uncaught SyntaxError: The requested module './auth.js' does not provide an export named 'logoutSession'	Причина: Скрипт ядра <code>app.js</code> пытается импортировать функцию выхода, у которой на стороне <code>auth.js</code> отсутствует директива <code>export</code>.  Фикс: Добавить явное объявление <code>export function logoutSession()</code> в коде управления сессиями.
 Вывод битых спецсимволов и кракозябр вида âš ĭ, на экране входа	Причина: Бэкэнд шлет сырые двухбайтовые эмодзи в ASCII/CP1251 заголовках ответа без явного UTF-8 флага веб-сервера.  Фикс: Навесить санитизацию строк на клиенте через регулярное выражение <code>.replace(/[\^x00-\x7Fa-яA-ЯёЁs\.,!\?-]/g, '').</code>

КОНТУР ИНТЕРФЕЙСНОЙ ДИАГНОСТИКИ И ДЕБАЖА UI УСПЕШНО ЗАКРЫТ

🛡️ Профиль ИБ-безопасности при проведении отладочных работ клиента:

- **Защита от XSS-инъекций через отладочный вывод ошибок (XSS Prevention):** При получении любых динамических текстов ошибок с бэкенда (свойства `data.detail` или `err.message`), фронтенд принудительно очищает их перед встраиванием в страницу. Вместо вызова небезопасных методов сырого рендеринга, код прогоняется через ИБ-выпрямитель санитизации спецсимволов. Это на 100% исключает риски внедрения вредоносных HTML/JS скриптов через подмену ответов базы данных или полей сообщений сетевых таймаутов.
- **Изоляция сессионных токенов при крахах рантайма:** При любых критических сбоях на стороне интерфейса, сессионные JWT-ключи остаются изолированы в защищенном хранилище `localStorage` и не вымываются во внешние логи.

=====
=====